

Методичка 17. Абстрактные классы и SOLID для школьников

1. Зачем нужна эта тема

Абстрактные классы и SOLID нужны, когда программа становится большой и в ней появляется много похожих классов.

Например, программа может принимать оплату разными способами:

- банковской картой;
- наличными;
- криптовалютой;
- бонусами.

У каждого способа оплаты разная внутренняя логика. Но для основной программы важно одно: любой способ оплаты должен уметь выполнить оплату.

Можно договориться, что у каждого класса оплаты будет метод:

```
pay ( amount )
```

Тогда основная программа сможет работать с разными способами оплаты одинаково.

Абстрактные классы помогают задать такой договор.

SOLID — это набор принципов, которые помогают писать код так, чтобы его было проще менять, расширять и не ломать.

Для школьников эту тему не нужно изучать как сложную теорию. Достаточно понять главную мысль: классы должны быть понятными, не слишком большими и не должны ломать друг друга.

2. Главная идея темы

Абстрактный класс — это класс-договор.

Он говорит: “Все наследники должны иметь такие-то методы”.

Например:

```
class PaymentSystem:
    def pay(self, amount):
        pass
```

Но обычный `pass` не заставляет наследников написать метод. Поэтому в Python используют `ABC` и `@abstractmethod`.

Абстрактный класс — класс, который задаёт общий интерфейс для наследников.

Интерфейс — набор методов, которые должен иметь объект.

`ABC` — базовый класс для создания абстрактных классов.

`@abstractmethod` — пометка метода, который наследник обязан реализовать.

SOLID — набор принципов хорошей структуры кода.

Полиморфизм — возможность работать с разными объектами одинаково, если у них есть общий набор методов.

3. Базовый синтаксис

Импорт ABC и abstractmethod

```
from abc import ABC, abstractmethod
```

Перед созданием абстрактного класса нужно импортировать `ABC` и `abstractmethod`.

Абстрактный класс

```
from abc import ABC, abstractmethod

class PaymentSystem(ABC):
    @abstractmethod
    def pay(self, amount):
        pass
```

Что происходит:

- `PaymentSystem` наследуется от `ABC` ;

- метод `pay` отмечен как абстрактный;
- все наследники должны реализовать `pay`.

Наследник

```
class CardPayment(PaymentSystem):  
    def pay(self, amount):  
        print("Оплата картой:", amount)
```

Класс `CardPayment` реализует обязательный метод `pay`.

Использование разных объектов одинаково

```
class CashPayment(PaymentSystem):  
    def pay(self, amount):  
        print("Оплата наличными:", amount)  
  
payments = [  
    CardPayment(),  
    CashPayment()  
]  
  
for payment in payments:  
    payment.pay(1000)
```

Основная программа не думает, какой именно способ оплаты используется. Она просто вызывает `pay`.

4. Разбор темы от простого к сложному

Шаг 1. Проблема без общего интерфейса

```
class CardPayment:  
    def pay_by_card(self, amount):  
        print("Карта:", amount)  
  
class CashPayment:  
    def cash_pay(self, amount):  
        print("Наличные:", amount)
```

Проблема: у классов разные названия методов.

Основной программе придётся писать много условий:

```
if type_payment == "card":
    payment.pay_by_card(1000)
elif type_payment == "cash":
    payment.cash_pay(1000)
```

Шаг 2. Единое название метода

```
class CardPayment:
    def pay(self, amount):
        print("Карта:", amount)

class CashPayment:
    def pay(self, amount):
        print("Наличные:", amount)
```

Теперь разные классы можно использовать одинаково:

```
payment.pay(1000)
```

Шаг 3. Абстрактный класс как договор

```
from abc import ABC, abstractmethod

class PaymentSystem(ABC):
    @abstractmethod
    def pay(self, amount):
        pass
```

Этот класс говорит: любой способ оплаты обязан иметь метод `pay`.

Шаг 4. Реализация наследников

```
class CardPayment(PaymentSystem):
    def pay(self, amount):
        print("Оплата картой:", amount)

class CashPayment(PaymentSystem):
    def pay(self, amount):
        print("Оплата наличными:", amount)
```

Каждый наследник реализует обязательный метод по-своему.

Шаг 5. Полиморфизм

```
def make_payment(payment_system, amount):  
    payment_system.pay(amount)  
  
make_payment(CardPayment(), 1000)  
make_payment(CashPayment(), 500)
```

Функция `make_payment` работает с любым объектом, у которого есть метод `pay`.

Шаг 6. SRP — один класс, одна понятная ответственность

Плохо:

```
class User:  
    def save_to_file(self):  
        pass  
  
    def send_email(self):  
        pass  
  
    def check_password(self):  
        pass  
  
    def print_report(self):  
        pass
```

Класс делает слишком много разного.

Лучше:

```
class User:
    def check_password(self):
        pass

class UserRepository:
    def save_to_file(self, user):
        pass

class EmailService:
    def send_email(self, user):
        pass
```

Главная мысль: не нужно складывать всё в один огромный класс.

Шаг 7. OCP — открыто для расширения, закрыто для постоянного переписывания

Плохо:

```
def pay(payment_type, amount):
    if payment_type == "card":
        print("Оплата картой")
    elif payment_type == "cash":
        print("Оплата наличными")
```

Если добавить новый способ оплаты, нужно менять старую функцию.

Лучше:

```
class CryptoPayment(PaymentSystem):
    def pay(self, amount):
        print("Оплата криптовалютой:", amount)
```

Новый способ оплаты добавляется новым классом.

Шаг 8. DIP — зависеть от общего договора, а не от конкретного класса

Плохо:

```
def buy():
    payment = CardPayment()
    payment.pay(1000)
```

Функция жёстко привязана к оплате картой.

Лучше:

```
def buy(payment_system):
    payment_system.pay(1000)
```

Теперь в функцию можно передать любой способ оплаты.

Шаг 9. ISP и LSP простыми словами

ISP — не заставлять класс реализовывать методы, которые ему не нужны.

Плохо:

```
class Worker:
    def work(self):
        pass

    def eat(self):
        pass
```

Если `Robot` наследуется от `Worker`, метод `eat()` ему не подходит.

Лучше разделить:

```
class Workable:
    def work(self):
        pass

class Eatable:
    def eat(self):
        pass
```

LSP — наследник должен нормально заменять родителя.

Если программа ждёт объект `PaymentSystem`, любой его наследник должен уметь оплатить через `pay`.

5. Частые ошибки учеников

Ошибка 1. Делают абстрактный класс без ABC

Неправильно:

```
class PaymentSystem:
    def pay(self, amount):
        pass
```

Почему это слабый вариант:

Такой класс не заставляет наследников реализовать `pay`.

Правильно:

```
from abc import ABC, abstractmethod

class PaymentSystem(ABC):
    @abstractmethod
    def pay(self, amount):
        pass
```

Ошибка 2. Забывают `@abstractmethod`

Неправильно:

```
class PaymentSystem(ABC):
    def pay(self, amount):
        pass
```

Почему это ошибка:

Метод не считается обязательным для наследников.

Правильно:

```
class PaymentSystem(ABC):
    @abstractmethod
    def pay(self, amount):
        pass
```

Ошибка 3. Наследник не реализует обязательный метод

Неправильно:


```
class CardPayment(PaymentSystem):  
    pass
```

Почему это ошибка:

Класс обязан реализовать `pay`.

Правильно:

```
class CardPayment(PaymentSystem):  
    def pay(self, amount):  
        print("Оплата картой:", amount)
```

Ошибка 4. Разные названия методов у похожих классов

Неправильно:

```
class CardPayment:  
    def pay_card(self, amount):  
        pass  
  
class CashPayment:  
    def pay_cash(self, amount):  
        pass
```

Почему это неудобно:

Основной программе сложно работать с такими объектами одинаково.

Правильно:

```
class CardPayment:  
    def pay(self, amount):  
        pass  
  
class CashPayment:  
    def pay(self, amount):  
        pass
```

Ошибка 5. Один класс делает всё

Неправильно:

```
class App:
    def input_data(self):
        pass

    def save_data(self):
        pass

    def send_email(self):
        pass

    def print_report(self):
        pass
```

Почему это плохо:

Класс слишком большой и отвечает за разные задачи.

Правильно:

```
class InputService:
    pass

class SaveService:
    pass

class EmailService:
    pass
```

Ошибка 6. Слишком много условий вместо классов

Неправильно:

```
if payment_type == "card":
    print("Карта")
elif payment_type == "cash":
    print("Наличные")
elif payment_type == "crypto":
    print("Крипта")
```

Почему это плохо:

При добавлении нового варианта нужно менять старый код.

Правильно:

```
payment.pay(amount)
```

А конкретная логика находится в классе способа оплаты.

Ошибка 7. Создают абстракции слишком рано

Неправильно:

```
class AbstractBaseMegaService(ABC):  
    pass
```

Почему это плохо:

Если в программе всего один класс и нет реальной необходимости, абстракция только усложняет код.

Правильно:

Сначала писать простой класс. Абстрактный класс добавлять тогда, когда появилось несколько похожих классов с общим интерфейсом.

6. Мини-примеры для закрепления

Пример 1. Абстрактный класс животного

```
from abc import ABC, abstractmethod  
  
class Animal(ABC):  
    @abstractmethod  
    def speak(self):  
        pass
```

Закрепляется `ABC` и `@abstractmethod`.

Пример 2. Наследник

```
class Cat(Animal):
    def speak(self):
        print("Мяу")

cat = Cat()
cat.speak()
```

Закрепляется реализация обязательного метода.

Пример 3. Несколько наследников

```
class Dog(Animal):
    def speak(self):
        print("Гав")

animals = [Cat(), Dog()]

for animal in animals:
    animal.speak()
```

Закрепляется полиморфизм.

Пример 4. Абстрактная оплата

```
class PaymentSystem(ABC):
    @abstractmethod
    def pay(self, amount):
        pass
```

Закрепляется общий интерфейс.

Пример 5. Оплата картой

```
class CardPayment(PaymentSystem):
    def pay(self, amount):
        print("Оплата картой:", amount)
```

Закрепляется конкретная реализация.

Пример 6. Функция зависит от интерфейса

```
def make_payment(payment_system, amount):
    payment_system.pay(amount)

make_payment(CardPayment(), 1000)
```

Закрепляется DIP в простом виде.

Пример 7. Разделение ответственности

```
class Report:
    def build_text(self):
        return "Отчёт"

class ReportPrinter:
    def print_report(self, report):
        print(report.build_text())
```

Закрепляется идея SRP.

7. Практические задания

Лёгкий уровень

1. Создать абстрактный класс `Animal` с методом `speak`.
2. Создать классы `Cat` и `Dog`, которые реализуют `speak`.
3. Создать список животных и вызвать у каждого `speak`.
4. Создать абстрактный класс `Transport` с методом `move`.
5. Создать классы `Car` и `Bike`, которые реализуют `move`.

Средний уровень

1. Создать абстрактный класс `PaymentSystem` с методом `pay`.
2. Создать классы `CardPayment`, `CashPayment`, `BonusPayment`.
3. Написать функцию `make_payment(payment_system, amount)`.
4. Создать список разных способов оплаты и вызвать у каждого `pay`.
5. Разделить большой класс `UserManager` на 2–3 маленьких класса по ответственности.

Повышенный уровень

1. Создать абстрактный класс `NotificationSender` с методом `send`.
2. Реализовать `EmailSender`, `SmsSender`, `TelegramSender`.
3. Написать функцию, которая принимает любой `NotificationSender` и отправляет сообщение.
4. Придумать пример, где нарушение SRP делает класс слишком сложным.
5. Придумать пример, где новый класс можно добавить без изменения старой функции.

8. Контрольные вопросы

1. Что такое абстрактный класс?
2. Для чего нужен `ABC`?
3. Для чего нужен `@abstractmethod`?
4. Что такое интерфейс?
5. Почему важно одинаковое название методов у похожих классов?
6. Что такое полиморфизм?
7. Почему функция может работать с разными объектами одинаково?
8. Что означает SRP простыми словами?
9. Что означает OCP простыми словами?
10. Что означает DIP простыми словами?
11. Почему не стоит делать один огромный класс?
12. Когда абстрактный класс действительно полезен?

9. Краткий конспект в конце

Главная идея темы: абстрактный класс задаёт договор для наследников, а SOLID помогает делать код понятнее и устойчивее к изменениям.

Основная конструкция:

```
from abc import ABC, abstractmethod

class PaymentSystem(ABC):
    @abstractmethod
    def pay(self, amount):
        pass
```

Наследник:

```
class CardPayment(PaymentSystem):
    def pay(self, amount):
        print("Оплата картой:", amount)
```

Полиморфизм:

```
def make_payment(payment_system, amount):
    payment_system.pay(amount)
```

Важно запомнить:

- абстрактный класс — это договор;
- `@abstractmethod` делает метод обязательным;
- наследник должен реализовать абстрактный метод;
- одинаковый интерфейс позволяет работать с разными объектами одинаково;
- SRP — один класс отвечает за одну понятную задачу;
- OCP — добавлять новое лучше без постоянного переписывания старого;
- DIP — лучше зависеть от общего интерфейса, а не от конкретного класса.

10. Что ученик должен уметь после методички

После этой темы ученик должен уметь:

- создавать абстрактный класс;
- использовать `ABC` ;
- использовать `@abstractmethod` ;
- создавать наследников абстрактного класса;
- реализовывать обязательные методы;
- объяснять идею интерфейса;
- использовать полиморфизм на простых примерах;
- понимать базовые идеи SOLID;
- видеть, когда класс делает слишком много;
- писать более гибкую структуру программы.